



Solving Recurrences by Unrolling

Substitute the equation into itself, find the pattern, stop at the base case, add it up

Session 9 of 24 · Friday 16 October 2026 · 8:30 – 9:50 AM

· Prof. Guillaume Sanje Mpacko, Ph.D.

SESSION 9 OF 24

Solving Recurrences by Unrolling

Friday, 16 October 2026 · 8:30 – 9:50 AM

LEARNING OBJECTIVES

- ✓ Solve a recurrence by unrolling it k times and identifying the general term.
- ✓ Determine the value of k at which the base case is reached.
- ✓ Derive the cost of insertion sort and of the Towers of Hanoi from their recurrences.
- ✓ Explain why subtracting $(n-1)$ and dividing $(n/2)$ produce completely different behaviour.
- ✓ Confirm a solution by substitution, and recognise the classic error in that method.

FROM SESSION 8

We can now write $T(n) = T(n-1) + 1$, $T(n) = T(n/2) + 1$, $T(n) = 2T(n/2) + n$ straight off the code. An equation is not an answer. Today we turn these into closed forms — by the most elementary method there is.

1 · The Method

12 min

1

Substitute the recurrence into itself, two or three times. Do not simplify prematurely — you are looking for a pattern, and simplification hides it.

2 **Generalise:** write the expression after k substitutions.

3 **Stop:** find the value of k that reaches the base case.

4 **Add up** what has accumulated, and read off Θ .

Step 3 is where the two families separate

If the recurrence *subtracts* — $T(n-1)$ — then after k steps the argument is $n - k$, and the base case $n = 0$ is reached at $k = n$. If it *divides* — $T(n/2)$ — then after k steps the argument is $n/2^k$, and the base case is reached at $k = \log_2 n$. n steps versus $\log n$ steps: the whole difference between the two families is in this one line.

2 • Four Worked Examples 30 min

$$1 \cdot T(N) = T(N-1) + 1, \quad T(0) = 1$$

$$T(n) = T(n-1) + 1$$

$$= [T(n-2) + 1] + 1 = T(n-2) + 2$$

$$= T(n-3) + 3 = \dots = T(n-k) + k$$

$$\text{Base case at } k = n: T(n) = T(0) + n = n + 1.$$

$T(n) = \Theta(n)$ — recursive summing costs the same as a loop. $\square$

$$2 \cdot T(N) = T(N/2) + 1, \quad T(1) = 1 \quad \text{— BINARY SEARCH}$$

$$T(n) = T(n/2) + 1 = T(n/4) + 2 = T(n/8) + 3 = \dots = T(n/2^b) + k$$

$$\text{Base case when } n/2^b = 1, \text{ that is } k = \log_2 n.$$

$$T(n) = T(1) + \log_2 n = \log_2 n + 1.$$

$T(n) = \Theta(\log n)$. Chantal's sixteen questions, now derived. $\square$

PUT EXAMPLES 1 AND 2 SIDE BY SIDE

Identical glue — constant work outside the call. Identical number of calls — one. The *only* difference is $n-1$ versus $n/2$, and the answers are $\Theta(n)$ and $\Theta(\log n)$. The size reduction, not the amount of glue, decided everything here.

$3 \cdot T(N) = T(N-1) + N$, $T(0) = 1$ — INSERTION SORT

$$T(n) = T(n-1) + n = T(n-2) + (n-1) + n = T(n-3) + (n-2) + (n-1) + n$$

After k steps: $T(n) = T(n-k) + (n-k+1) + \dots + n$.

At $k = n$: $T(n) = T(0) + 1 + 2 + \dots + n = 1 + n(n+1)/2$.

$T(n) = \Theta(n^2)$ — the arithmetic series from Session 5, arriving by a completely different route. $\square$

$4 \cdot T(N) = 2T(N-1) + 1$, $T(0) = 1$ — TOWERS OF HANOI

$$T(n) = 2T(n-1) + 1$$

$$= 2[2T(n-2) + 1] + 1 = 4T(n-2) + 2 + 1$$

$$= 8T(n-3) + 4 + 2 + 1 = \dots = 2^b T(n-k) + (2^b - 1)$$

At $k = n$: $T(n) = 2^n \cdot 1 + 2^n - 1 = 2^{n+1} - 1$.

$T(n) = \Theta(2^n)$. $\square$

READ THE TWO KNOBS, EVERY TIME

How much smaller? $n-1$ gives n levels; $n/2$ gives $\log n$ levels.

How many calls? The coefficient in front of T decides how fast work multiplies as you descend.

Two calls on **half** the input is excellent (we will get $\Theta(n \log n)$ from it). Two calls on **$n-1$** is fatal. Same number of calls; different knob.

3 • The Two Algorithms Behind the Equations 20 min

3.1 Insertion sort — where $T(n) = T(n-1) + n$ comes from

Insertion sort is how most people sort a hand of playing cards: keep a sorted run on the left, and slide each new card leftwards until it lands in the right place. Sorting n cards means sorting the first $n-1$ and then inserting one more, which costs up to n comparisons. That sentence *is* the recurrence.

PY INSERTION_SORT

$\Theta(1)$ auxiliary space — sorts in place

```
def insertion_sort(A):
    """Sort in place. A[0:i] is always sorted when iteration i begins."""
    for i in range(1, len(A)):
        key = A[i]
        j = i - 1
```

```

while j >= 0 and A[j] > key:    # slide bigger elements one place right
    A[j + 1] = A[j]
    j -= 1
A[j + 1] = key                 # drop the key into the gap
return A

```

THE INVARIANT IS THE CORRECTNESS PROOF

At the start of every iteration, `A[0:i]` holds the first i elements in sorted order. The inner loop restores that for $i+1$. When the loop ends, $i = \text{len}(A)$ and the array is sorted. A property that is true at the start, preserved by each step, and useful at the end is the proof. Nothing further is required.

Case	Input	Inner loop	Bound
Best	already sorted	the <code>while</code> test fails at once, every time	$\Theta(n)$
Worst	sorted in reverse	every element slides all the way	$\Theta(n^2)$
Average	random order	each element slides about half way	$\Theta(n^2)$

That best case is not a curiosity. Insertion sort is genuinely the fastest algorithm we will meet on input that is small or nearly sorted — which is why real sorting libraries keep it, as we will see on 23 October.

3.2 Towers of Hanoi — where $T(n) = 2T(n-1) + 1$ comes from

Move a stack of n discs from peg A to peg C, never placing a larger disc on a smaller one. The recursive idea is one sentence: move the top $n-1$ out of the way, move the big one, move the $n-1$ back on top.

PY HANOI

```

def hanoi(n, source, target, spare, moves):
    """Record the sequence of moves. Returns nothing; appends to `moves`."""
    if n == 0:
        return
    hanoi(n - 1, source, spare, target, moves)    # get n-1 discs out of the way
    moves.append((source, target))                # move the largest disc: 1 move
    hanoi(n - 1, spare, target, source, moves)    # put them back on top

for n in range(1, 21):
    moves = []
    hanoi(n, 'A', 'C', 'B', moves)
    print(n, len(moves))

```

EXACTLY $2^n - 1$, AS THE RECURRENCE PREDICTED

n	moves
1	1
2	3
3	7
4	15
5	31
10	1,023
15	32,767
20	1,048,575

The legend, and the arithmetic

The temple legend has 64 golden discs. At one move per second, $2^{64} - 1$ moves takes about **585 billion years**. The algorithm is optimal — no procedure can do it in fewer moves — so this is not a bad algorithm. It is a problem whose *answer* is exponentially large, and no cleverness can shrink it. Distinguishing “my algorithm is slow” from “this problem is hard” is a theme we return to at the end of the course.

4 · Confirming an Answer by Substitution 12 min

Unrolling produces an answer by pattern-spotting, which is not quite a proof. Substitution makes it one: guess the closed form, then verify it by induction.

CLAIM · $T(N) = T(N-1) + N$ IS $O(N^2)$

Guess $T(n) \leq c n^2$ for some constant c and all $n \geq 1$.

Assume it holds for $n-1$: $T(n-1) \leq c(n-1)^2$.

Then $T(n) \leq c(n-1)^2 + n = cn^2 - 2cn + c + n$.

This is $\leq cn^2$ provided $-2cn + c + n \leq 0$, which holds for $c \geq 1$ and $n \geq 1$. The guess survives. $\square$

The classic substitution error

You must recover **exactly** the form you assumed — not something weaker. Arriving at “ $\leq cn^2 + n$ ” and declaring victory is invalid. That leftover n , accumulated across all the levels, is precisely the difference between one complexity class and the next. If you cannot absorb the leftover, your guess was wrong; strengthen it and try again.

SESSION 9 TAKEAWAY

Unrolling is substitution plus patience: expand, generalise to k , find the k that hits the base case, and sum. Subtraction gives n levels; division gives $\log n$ levels. The coefficient in front of T decides whether the work multiplies as you descend. Insertion sort is $\Theta(n^2)$ and Hanoi is $\Theta(2^n)$ for reasons you can now derive rather than recall.

IN-CLASS EXERCISES (15 min — work in pairs)

1. Unroll and solve: (a) $T(n) = T(n-1) + 2n$; (b) $T(n) = T(n/2) + n$; (c) $T(n) = T(n-2) + 1$; (d) $T(n) = 3T(n-1) + 1$.
2. Unroll $T(n) = T(n-1) + \log n$. (Hint: a sum of logs is the log of a product, and that product is n factorial.)
3. Trace `insertion_sort([5, 2, 9, 1, 6])` and write down the array after each outer iteration. How many slides in total?
4. Why does unrolling become unpleasant for $T(n) = 2T(n/2) + n$? Attempt two substitutions and say precisely what goes wrong. (This motivates Tuesday.)
5. Verify by substitution that $T(n) = T(n/2) + 1$ is $O(\log n)$.

Homework • due Tuesday 20 October

1. Implement `insertion_sort`. Verify against `sorted()` on 500 random lists, then time it at $n = 1000, 2000, 4000$ on random, sorted, and reverse-sorted input. Three columns — explain each one using §3.1.
2. Implement `hanoi` and confirm the $2^n - 1$ count for $n = 1 \dots 20$. Then print the actual moves for $n = 3$ and check by hand that they are legal.
3. Solve by unrolling, showing every step: $T(n) = 2T(n-1) + n$ with $T(0) = 1$.
4. Solve $T(n) = T(n/3) + T(2n/3) + n$ by unrolling as far as you can, and explain why the method struggles here. Do not force it — the difficulty is the exercise.
5. Verify by substitution that $T(n) = 2T(n-1) + 1$ is $O(2^n)$.
6. Reading: CLRS §4.4. Half a page on why unrolling and substitution are complementary rather than alternatives.

Next session · Tuesday 20 October: Recursion trees. When unrolling runs out of patience, we draw the recursion instead — and one question about the picture settles every divide-and-conquer recurrence in this course.